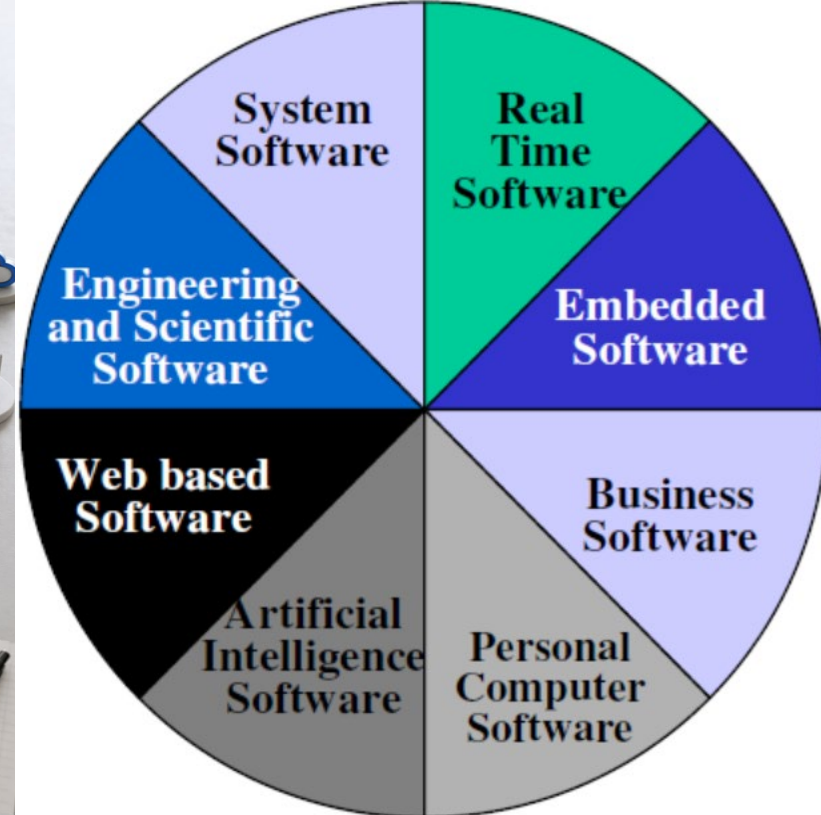




Software Engineering(SE)



Software Engineering





Software Engineering(SE)





Software Engineering(SE)



Why study Software Engineering?

- ✓ Understand how complex, scalable systems are built.
- ✓ Support innovation in logic and algorithm design.
- ✓ Improve system performance and user experience.
- ✓ Create value through automation and new features.
- ✓ Prepare for a high-demand, creative career in the digital world.

SOFTWARE ENGINEERING (SE)

1. What is SE?



SE is the disciplined application of engineering principles to the design, development, maintenance, and testing of software to ensure reliability, scalability, and quality.

2. Key Role of SE



Defines software architecture and standards.



Writes and reviews clean, efficient code.



Implements rigorous testing and QA processes.



Manages deployment and system reliability.

3. Examples of SE Focus



Microservices Architecture



Real-time Data Pipelines



Machine Learning Integration



Secure APIs



Distributed Databases



User Interface Systems

4. Benefits of Studying SE



Very high global demand for engineers.



Diverse career paths (Web, Mobile, Game, Data).



Focuses on logic, creativity, and problem-solving.



Essential for building impactful digital products.



Fosters continuous learning and technological leadership.

In short:

Software Engineering builds the robust systems that power our digital world efficiently and safely.



"Software engineering is the foundation of digital Infrastructure. It ensures the systems we depend on are built to last." – Lim Lyheng



Software Engineering(SE)



Why Software Engineering is Important?

- ✓ Create resilient, secure digital infrastructure.
- ✓ Enable seamless global communication and collaboration.
- ✓ Drive innovation in critical sectors like healthcare and finance.
- ✓ Safely manage and protect sensitive data and privacy.
- ✓ Construct systems that can handle massive scale efficiently.

► IMPACT OF SOFTWARE ENGINEERING (SE)

1. Critical Systems SE Enables

- Health Information Systems (EMRs, telemedicine)
- Financial Networks (banking, stock exchanges)
- Urban Infrastructure (traffic management, power grids)
- Data Privacy and Security Systems

2. SE as an Economic & Innovation Driver

- Accelerating Automation and Efficiency
- Powering AI and Machine Learning Innovations
- Creating a Digital Economy and New Jobs
- Enabling Advanced Scientific Research and Exploration

3. Real-world Examples of SE Importance

- Large-Scale Distributed Cloud Platforms (e.g., major service providers)
- Critical Global Communication Systems
- Accessible and Secure Digital Commerce Solutions
- Precision-Guided Healthcare Solutions

4. The Long-term Value of SE Discipline

- Ensures Reliability and System Integrity
- Provides Future-Proof Scalability and Extensibility
- Fosters Ethical and Accessible Technology Development
- Builds Trust and Dependability in Digital Services

In short:

Software Engineering is the backbone of modern society, transforming complex global challenges into reliable, secure, and scalable solutions that improve lives daily.



"Software engineering is the art of building a dependable digital world. It is the invisible force that makes modern life functional, secure, and full of possibility." – Lim Lyheng





Software Engineering(SE)



Why Software Engineering important?

Four reasons why software engineering is essential

1. **Ensures the development** of high-quality, reliable, and efficient software solutions.
2. **Minimizes potential risks and issues** associated with software development.
3. **Optimizes** resources, reducing costs and development time.
4. **Facilitates communication and collaboration** among team members.



Software Engineering(SE)



Course Learning Objectives (CLOs)

1.Understand Software Engineering Principles Explain fundamental concepts, models, and best practices in software development.	7.Manage Software Projects Plan, schedule, and manage resources, risks, and timelines in software projects.
2.Analyze and Define Requirements Gather, analyze, and document user and system requirements effectively.	8.Use Modern Tools and Technologies Apply current development tools, frameworks, and version control systems.
3.Design Software Systems Create structured and scalable software designs using appropriate methodologies and tools.	9.Evaluate Software Performance and Security Assess system performance and implement security measures.
4.Apply Development Methodologies Use models such as Agile, Waterfall, and Scrum in real-world projects.	10.Work Effectively in Teams Collaborate with others and communicate effectively in a development environment.
5.Develop High-Quality Software Write efficient, reliable, and maintainable code following coding standards.	11.Apply Ethical and Professional Practices Understand legal, ethical, and professional responsibilities in software development.
6.Perform Software Testing & Quality Assurance Apply testing techniques to ensure software correctness and reliability.	12.Engage in Lifelong Learning Adapt to new technologies and continuously improve technical skills.



Software Engineering(SE)



Learning Outcome?

1. Ability to **develop future software solutions**
2. Ability to **counsel CEOs and political decision makers** on investments in new software solutions based on your ability to assess user needs and technological possibilities
3. Ability to **head the development and implementation of IT based** on a thorough knowledge of software and its strengths and weaknesses in relation to a specific company.
4. Ability to **work effectively in a team** to analyze the requirements of a complex software system, and to create a design that satisfies these requirements
5. Ability to **evaluate requirements** for a complex software system
6. Ability to **explain key aspects of models** and processes for designing a complex software system



Software Engineering(SE)



Software Engineering ebooks



Part 1	Introduction to Software Engineering	15
Chapter 1	Introduction	17
Chapter 2	Software processes	43
Chapter 3	Agile software development	72
Chapter 4	Requirements engineering	101
Chapter 5	System modeling	138
Chapter 6	Architectural design	167
Chapter 7	Design and implementation	196
Chapter 8	Software testing	226
Chapter 9	Software evolution	255
Part 2	System Dependability and Security	283
Chapter 10	Dependable systems	285
Chapter 11	Reliability engineering	306
Chapter 12	Safety engineering	339
Chapter 13	Security engineering	373
Chapter 14	Resilience engineering	408
Part 3	Advanced Software Engineering	435
Chapter 15	Software reuse	437
Chapter 16	Component-based software engineering	464
Chapter 17	Distributed software engineering	490
Chapter 18	Service-oriented software engineering	520
Chapter 19	Systems engineering	551
Chapter 20	Systems of systems	580
Chapter 21	Real-time software engineering	610
Part 4	Software Management	639
Chapter 22	Project management	641
Chapter 23	Project planning	667
Chapter 24	Quality management	700
Chapter 25	Configuration management	730

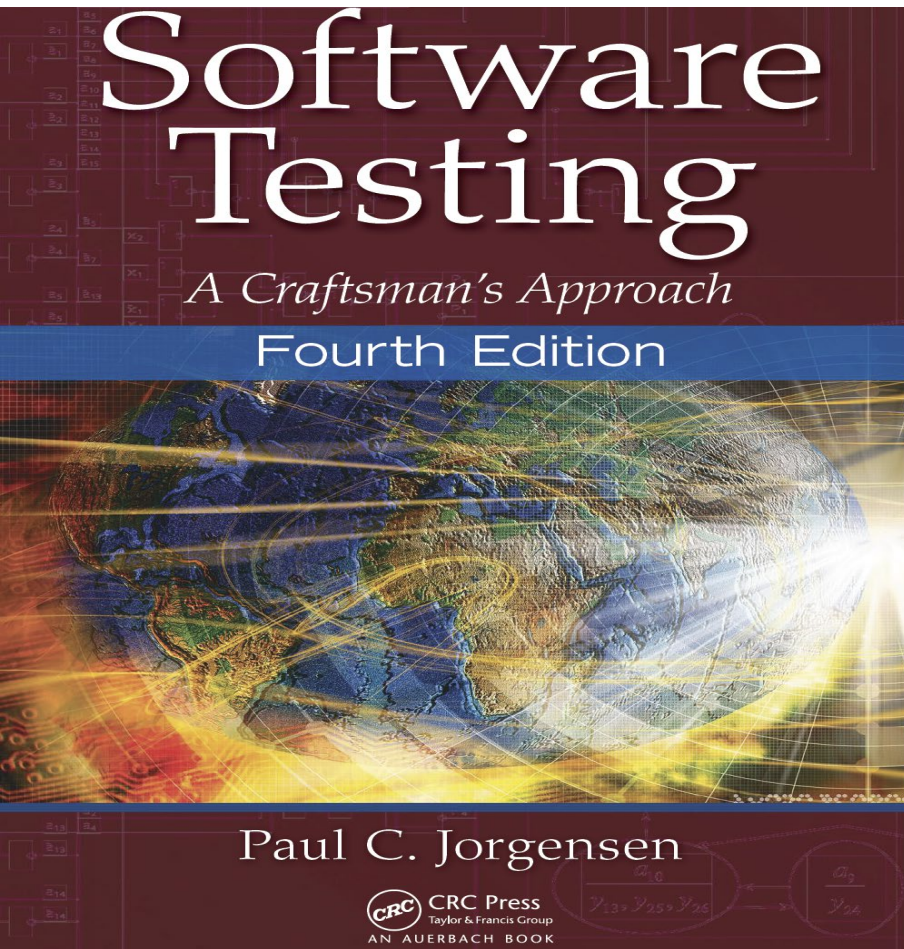




Software Engineering(SE)



Software Engineering ebooks



- Chapter 2: Examples
- Chapter 3: Discrete Math for Testers
- Chapter 4: Graph Theory for Testers
- Chapter 5: Boundary Value Testing
- Chapter 6: Equivalence Class Testing
- Chapter 7: Decision Table-Based Testing
- Chapter 8: Path Testing
- Chapter 9: Data Flow Testing
- Chapter 10: Retrospective on Unit Testing
- Chapter 11: Life Cycle-Based Testing
- Chapter 12: Model-Based Testing
- Chapter 13: Integration Testing
- Chapter 14: System Testing
- Chapter 15: Object-Oriented Testing
- Chapter 16: Software Complexity**
- Chapter 17: Model-Based Testing for Systems of Systems
- Chapter 18: Exploratory Testing
- Chapter 19: Test-Driven Development
- Chapter 20: A Closer Look at All Pairs Testing
- Chapter 21: Evaluating Test Cases

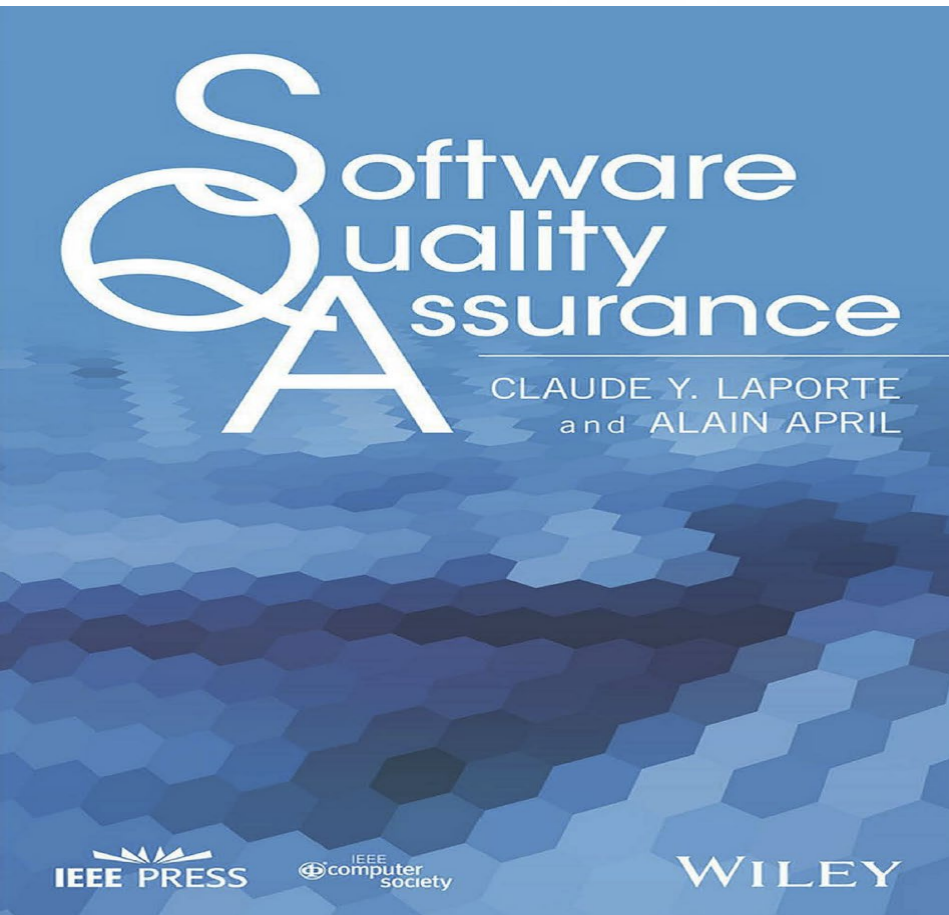




Software Engineering(SE)



Software Engineering ebooks



1. Software Quality Fundamentals

1.1	Introduction	1
1.2	Defining Software Quality	2
1.3	Software Errors, Defects, and Failures	4
1.3.1	Problems with Defining Requirements	10
1.3.2	Maintaining Effective Communications Between Client and Developer	13
1.3.3	Deviations from Specifications	14
1.3.4	Architecture and Design Errors	15
1.3.5	Coding Errors	15
1.3.6	Non-Compliance with Current Processes/Procedures	16
1.3.7	Inadequate Reviews and Tests	17
1.3.8	Documentation Errors	17
1.4	Software Quality	19
1.5	Software Quality Assurance	20
1.6	Business Models and the Choice of Software Engineering Practices	22
1.6.1	Description of the Context	23
1.6.2	Anxiety and Fear	24
1.6.3	Choice of Software Practices	25
1.6.4	Business Model Descriptions	25
1.6.5	Description of Generic Situational Factors	26
1.6.6	Detailed Description of Each Business Model	27
1.7	Success Factors	32





Software Engineering(SE)



Software Engineering ebooks



A Guide to the SCRUM BODY OF KNOWLEDGE (SBOK™ GUIDE) 2016 Edition

A Comprehensive Guide to Deliver Projects using Scrum

3. ORGANIZATION	39
3.1 Introduction	39
3.2 Roles Guide	40
3.3 Scrum Project Roles	40
3.3.1 Core Roles	41
3.3.2 Non-core Roles	42
3.4 Product Owner	43
3.4.1 Voice of the Customer (VOC)	45
3.4.2 Chief Product Owner	45
3.5 Scrum Master	45
3.5.1 Chief Scrum Master	47
3.6 Scrum Team	47
3.6.1 Personnel Selection	49
3.6.2 Scrum Team Size	49
3.7 Scrum in Projects, Programs, and Portfolios	50
3.7.1 Definition of Project, Program, and Portfolio	50
3.7.2 Scrum in Projects	51
3.7.3 Scrum in Portfolios and Programs	53
3.7.4 Maintaining Stakeholder Involvement	55
3.8 Summary of Responsibilities	56
3.9 Scrum vs. Traditional Project Management	57
3.10 Popular HR Theories and their Relevance to Scrum	58
3.10.1 Tuckman's Model of Group Dynamics	58
3.10.2 Conflict Management	59
3.10.3 Conflict Management Techniques	59
3.10.4 Leadership Styles	61
3.10.5 Maslow's Hierarchy of Needs Theory	63
3.10.6 Theory X and Theory Y	64





Software Engineering(SE)



Chapter Outline

1. What is Software Engineering?
2. The Applications of Software Engineering in Practices
3. Why need to study Software Engineering?
4. Tops skills for Software Engineer
5. The characteristics of Good Software Engineer
6. The Goals of Software Engineering
7. The Characteristics of Good Software
8. The Key Challenges of Software Engineer
9. The Evolution of Software Engineering
10. Why Software Development is hard?
11. Code of Ethics and Professional Practices of Software Engineer



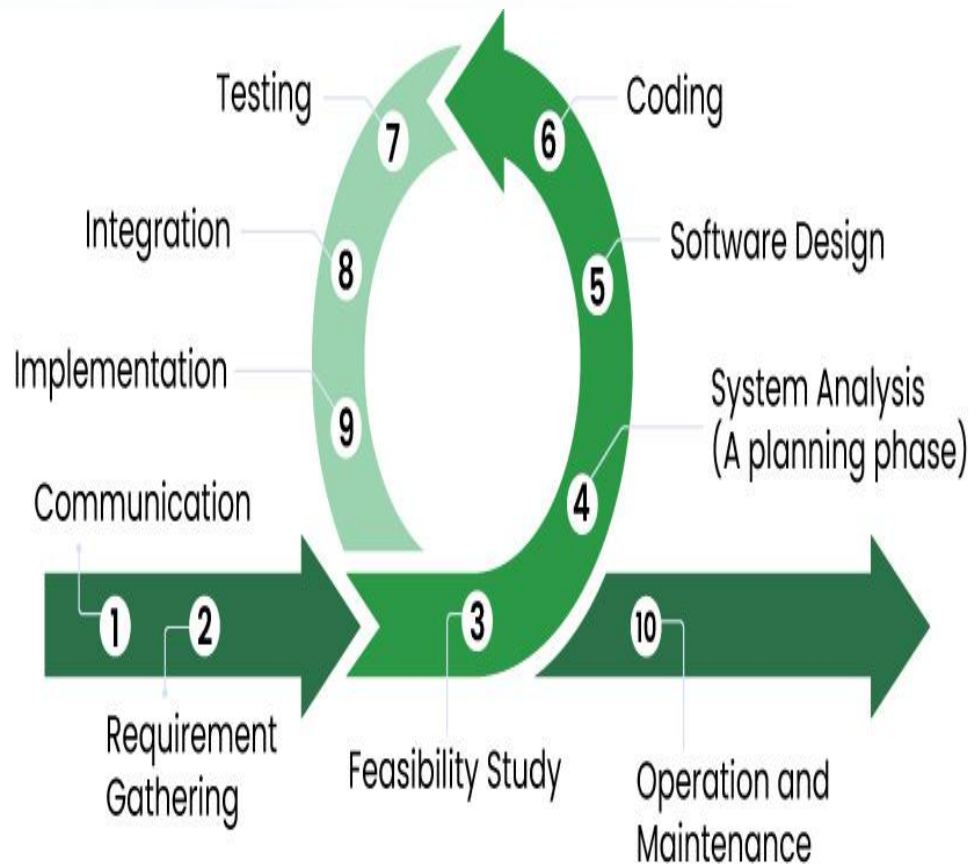
Software Engineering(SE)



1. What is Software Engineering?

Software engineering is an **engineering approach** to software development by **developing efficient and reliable software products** to

- ❖ meet the **clients' needs**
- ❖ and **user satisfactions**.





Software Engineering(SE)



Software Engineering Overview

- ❖ **Software** = Code + Libraries+ docs
- ❖ **A program** = is an executable code
- ❖ **Software Product**= specific requirement (HRMIS)
- ❖ **Engineering** = well-defined, scientific principles , methods.
- ❖ **Software engineering** = developing efficient and reliable software product using engineering mechanism.





Software Engineering(SE)



1. What is Software engineering now?

Software engineering is an engineering discipline that is concerned with all aspects of software production which is

- Robust/Bug-free software
- deliver on time
- **within budget plan**
- satisfy client's requirements ...



2.The Application of SE in Practice

Software Engineering(SE)



1.Mobile applications: Developing apps for smartphones and tablets, including productivity tools, games, and social media platforms.

2.Web applications: Creating web-based software for e-commerce, content management systems, social networks, and other online services.

3.Operating systems and system software: System developers design and develop operating systems, drivers, and utilities for computers, servers, and embedded systems.

4.Enterprise software: Building business applications, such as customer relationship management (CRM), enterprise resource planning (ERP), and human resources management systems (HRMS).

5.Scientific and research software: software engineers craft software for scientific research, data analysis, simulation, and visualization in fields like physics, biology, and astronomy.

6.Artificial intelligence and machine learning: Software engineers design and develop algorithms, models, and software tools for tasks like natural language processing, computer vision, and recommendation systems.

7.Internet of Things (IoT) and embedded systems: Creating software for connected devices, sensors, and smart appliances in industries like home automation, healthcare, and transportation.

8.Network control systems: Systems developers design specialized systems designed to monitor, manage, and control communication networks and their components.

9.Video games and multimedia applications: Being a software developer, you design and develop interactive entertainment software, including video games, virtual reality (VR), and augmented reality (AR) experiences.

10.Cybersecurity: Building security tools and software to protect against cyber threats, including firewalls, antivirus software programs, and intrusion detection systems.

11.Cloud computing and distributed systems: Developing software for cloud platforms, infrastructure management, and distributed computing solutions.

12.Embedded Systems

Software integrated into hardware devices.

Examples: automotive systems, smart TVs, medical devices

Focus: real-time performance, efficiency, hardware



Software Engineering(SE)



Software Engineering is the most well-paid Job



Computer Science Jobs



Software Engineering(SE)



High demand Job across the Globe





Software Engineering(SE)



Career Opportunities?





Software Engineering(SE)

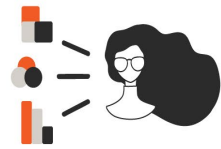


3.Why Need to Study Software Engineering?

1.Build complex systems effectively Helps develop large-scale applications like Amazon and Google Docs	6.Enhance career opportunities Opens jobs such as developer, architect, and project manager
2.Improve software quality Ensures software is reliable, secure, and easy to maintain	7.Build secure systems Protects data and prevents cyber attacks
3.Manage large projects Supports planning, teamwork, and use of Agile/Scrum methods	8.Solve real-world problems Used in healthcare, education, business, and more
4.Reduce development cost and time Early planning avoids expensive errors later	9.Keep up with modern technologies Covers cloud computing, AI, and mobile/web development
5.Ensure scalability and performance Enables systems like Telegram to handle millions of users	10.Develop professional skills Improves communication, teamwork, and problem-solving

A Day in the Life of a Software Engineer

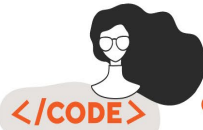
Entry-Level Software Engineer



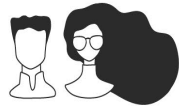
Implement **new features**

React
Ruby
Python

learn new **languages & frameworks**



Maintain **existing code**



Pair **programming**



Resolve **simple bugs / errors**

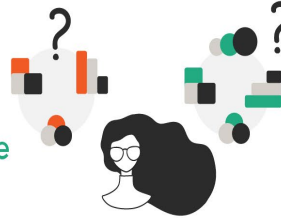


Refactor **code**

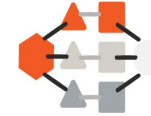


Practice **test driven development (TDD)**

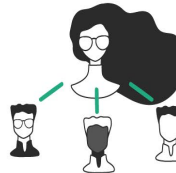
Decide which **framework & libraries** to use



Architect API **endpoints & data models**



Mentor junior engineers



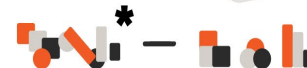
Attend sprint planning meetings with **product managers & designers**



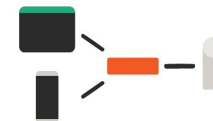
Code reviews



Refactor **legacy code**



Architect **scalable systems**



Senior Software Engineer





Software Engineering(SE)



4.Top Skills for Software Engineers

1✓

Problem Solving



2✓

Attention to Detail



3✓

Clear Communication



4✓

Continual Learning



5✓

Teamwork



6✓

Empathize with End Users





Software Engineering(SE)



5.The characteristics of good software engineer (Technical Ability + Strong Personal + Professional Qualities)

1. Strong Problem-Solving Skills Able to analyze complex problems and find efficient, logical solutions.	2. Solid Technical Knowledge Good understanding of programming languages, algorithms, data structures, and concepts in SE
3. Attention to Detail Writes accurate code, spot bugs or inconsistencies easily.	4. Good Communication Skills Can clearly explain ideas, collaborate with team members, and understand user requirements.
5. Teamwork and Collaboration Works effectively with others, including developers, designers, and stakeholders.	6. Adaptability and Continuous Learning Keeps up with new technologies, tools, and industry trends.
7. Time Management Able to meet deadlines and manage tasks efficiently.	8. Creativity and Innovation Finds new and better ways to solve problems or improve systems.
9. Patience and Persistence Willing to debug, test, and refine code without giving up easily.	10. Understanding of SE Development Process Familiar with development methodologies (Agile, Scrum, etc.) and best practices.



Software Engineering(SE)



6.The Key Challenges of Software Engineer

Software Engineering

+

People Skills

+

Artistry and Creativity

+

Craftsmanship and Competence

+

Domain Knowledge

- Requirements
- Design
- Coding
- Testing
- Deployment
- Documentation
- Maintenance
- Reengineering
- Configuration
- Management
- Processes
- Principles
- Patterns
- Practices

- Communication
- Social skills
- Teamwork
- Group dynamics
- Leadership
- Negotiation
- Systems Thinking

- Solving Problems
- Naming
- Constructive creativity
- Destructive creativity
- Pragmatism
- Intuition
- Innovation
- Experimentation
- Complexity Thinking
- Vision

- Clean Coding
- Teaching Yourself
- Lifelong Learning
- Business Knowledge
- Technical Knowledge
- Quality Thinking
- Time Management
- Risk Management
- Passion
- Discipline
- Culture
- Values
- Ethics

- Life Critical Products
- Security Models
- User Roles
- Terminology
- Archetype
- Standards
- Legislation
- Healthcare Knowledge
- Integration
- System Topology



Software Engineering(SE)



7.Goal of Software Engineering

To produce the reliable
high-quality software at low
cost.





8.Characteristics of good software

- A software product can be judged by what it offers and how well it can be used.
- This software must satisfy on the following grounds:
 1. *Operational*
 2. *Transitional*
 3. *Maintenance*
- Well-engineered and crafted software is expected to have the following characteristics:



8.Characteristics of good software

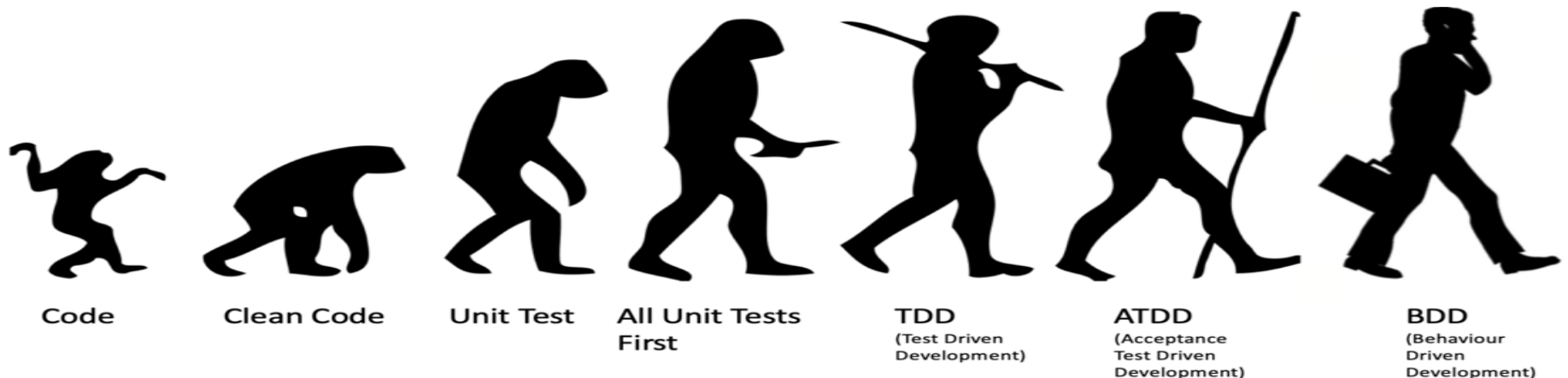
Operational	Transitional	Maintenance
This tells us how well the software works in operations. It can be measured on: 1. Budget 2. Usability 3. Efficiency 4. Correctness 5. Functionality 6. Dependability 7. Security	This aspect is important when the software is moved from one platform to another: 1. Portability 2. Interoperability 3. Reusability 4. Adaptability	This aspect briefs about how well the software has the capabilities to maintain itself in the ever-changing environment: 1. Modularity (<i>decomposable</i>) 2. Maintainability (<i>modifiable</i>) 3. Flexibility (<i>changeable</i>) 4. Scalability (<i>growable</i>)



Software Engineering(SE)



9. Software Engineering evolution





Software Engineering(SE)



9. Software Engineering evolution

1. **Continuing change:** continue to adapt to the real world changes, else less useful.
1. **Increasing complexity:** its complexity tends to increase unless work is done to maintain or reduce it.
2. **Continuing growth:** to resolve some business problem, its size of implementing the changes grows according to the lifestyle changes of the business.
3. **Reducing quality:** changing operational environment.
5. **Feedback systems:** multi-level feedback to be successfully modified or improved.
6. **Self-regulation:** self-regulating with the distribution of product and process
7. **Organizational stability:** The average effective global activity rate in an evolving over the lifetime of the product.



Software Engineering(SE)



Why Software is so expensive?

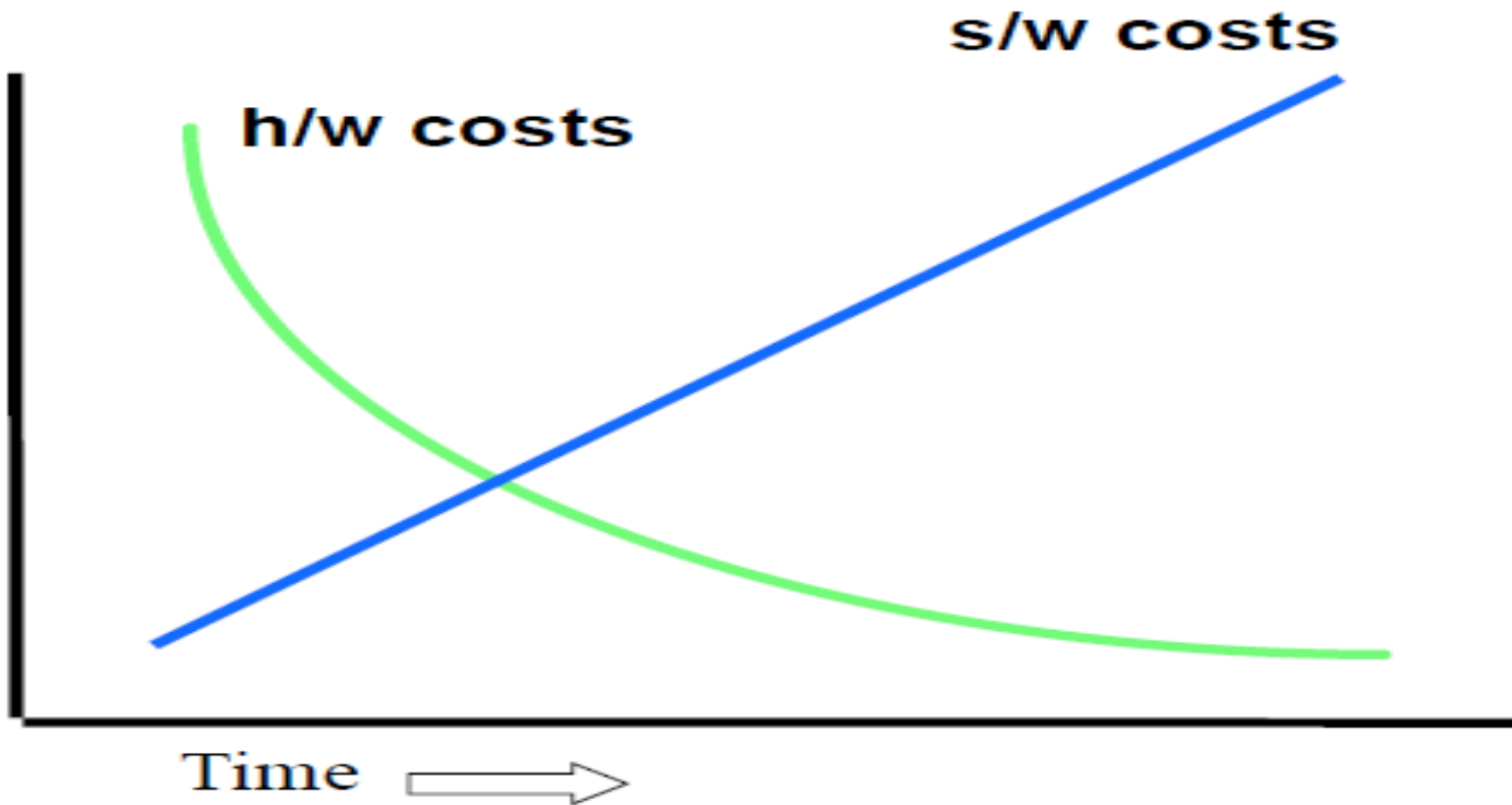
- ❖ Software is expensive!
- ❖ The major costs are salaries (your salaries)
- ❖ More and more demand from clients
- ❖ Every software project has a trade-off between:
 1. **Functionality**
 2. **Resources (cost)**
 3. **Timeliness**



Software Engineering(SE)



Hardware Cost Vs Software Cost?





Software Engineering(SE)



10. Why software development is so hard?

❖ Communication

- *Between customer and developer*
 - Poor problem definition is largest cause of failed software projects!
- *Within development team*
 - More people = more communication!
 - New programmers need training!

❖ Project characteristics

- Novelty!
- Changing requirements!
 - 5 x cost during development!
 - up to 100 x cost during maintenance!

❖ Hardware/software configuration

❖ Security requirements

❖ Real time requirements

❖ Reliability requirements



Software Engineering(SE)



Why software development is so hard?

❖ Personnel characteristics

- Ability!
- Prior experience!
- Communication skills!
- Team cooperation!
- Training!

❖ Facilities and resources!

- Identification
- Acquisition

❖ Management issues

- Realistic goals!
- Cost estimation!
- Scheduling!
- Resource allocation!
- Quality assurance!
- Version control!
- Contracts!



Software Engineering(SE)



11.Code of Ethics and Professional Practices of Software Engineer

1.Public Responsibility

- Act in the best interest of the public
- Ensure software does not harm users or society
- Consider safety, privacy and environmental impact

2.Client & Employer Responsibility

- Act in the best interest of clients and employers
- Maintain confidentiality of sensitive information
- Deliver work honestly and within agreed requirements

3.Product Quality

- Develop high-quality, reliable and secure software
- Perform proper testing and validation
- Avoid releasing defective or harmful software

4.Professional Judgement

- Make decisions based on honesty and integrity
- Avoid conflicts of interest
- Provide objective and truthful evaluations



Software Engineering(SE)



11.Code of Ethics and Professional Practices of Software Engineer

5.Management Responsibility

- Promote ethical practices in project management
- Ensure realistic deadlines and proper resource allocation
- Support a healthy and fair environment

6.Profession Integrity

- Uphold the reputation of the software engineering profession
- Follow standards, laws & regulations
- Avoid unethical behavior such as plagiarism or fraud

7.Colleague Responsibility

- Treat coworkers with respect and fairness
- Support and help team members
- Give proper credit for others 'work

8.Self-Improvement

- Continuously improve skills and knowledge
- Stay updated with new technologies and practices
- Accept feedback and learn from mistake



Software Engineering(SE)



End of Chapter 1

Thank You





Software Engineering(SE)

